Instance Render

ISM

Instanced Static Mesh Rendering
将相同的实例合并成一个 Draw Call 进行绘制，不同 API 有不同的接口

问题

- 需要确定哪些 Mesh 放进一个批次进行渲染，不可见实例仍然需要进入渲染
- 每个 Draw Call 只能传入一个实例，所有的渲染实例需要强制使用相同的 LOD，如果实例分布比较散，实例的 LOD 可能一直在高精度上
- ISM 不对实例做剔除，只对整体 ISM 做剔除

HISM

Hierarchical Instanced Static Mesh Rendering
主要思路是在 ISM 基础上对实例进行剔除，同时允许不同实例不同 LOD

UE 里使用一个类似 BVH 树的方式管理实例，包含两个主要结构

```
struct FClusterTree {
    TArray<FClusterNode> Nodes;           // 树节点数组
    TArray<int32> SortedInstances;        // 排序后的实例索引
    TArray<int32> InstanceReorderTable;   // 实例重排表
    int32 OutOcclusionLayerNum = 0;       // 遮挡层数量
};

struct FClusterNode {
    FVector3f BoundMin, BoundMax;         // 节点包围盒
    int32 FirstChild, LastChild;          // 子节点索引范围
    int32 FirstInstance, LastInstance;    // 实例索引范围
    FVector3f MinInstanceScale, MaxInstanceScale; // 实例缩放范围
};
```

树的构建

树的构建需要分两步

1. 按 BVH 构建的方式将实例分组，形成树的叶子节点
2. 根据这些结点自底向上构建 Cluster Tree
   不直接一次构建 Cluster Tree 是为了避免直接构建 N 叉树导致的空间划分问题，使用二分后聚类可以有效避免精度下降

叶子节点划分

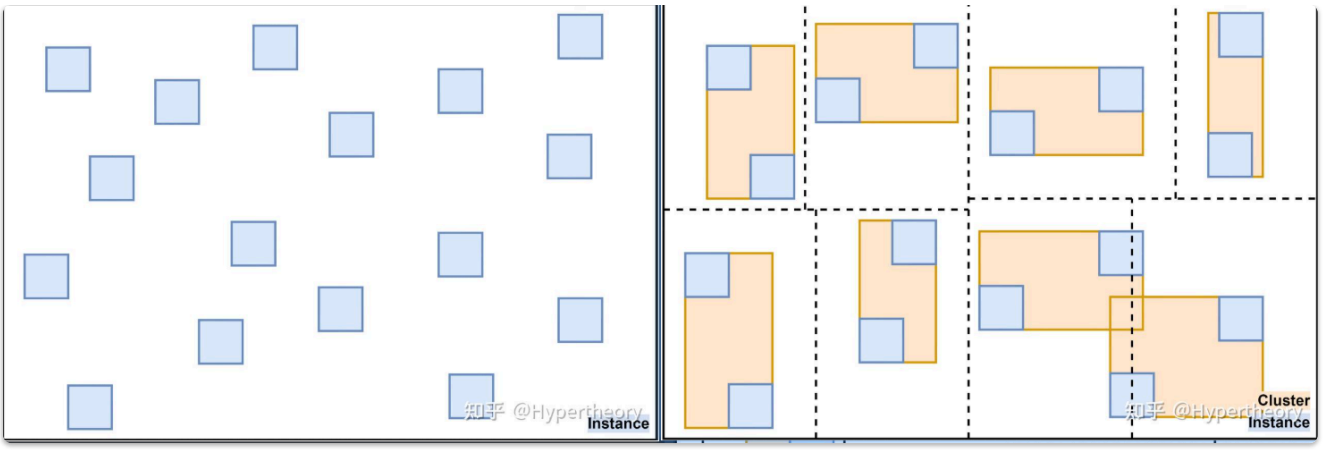

image-18.png

```
void
UHierarchicalInstancedStaticMeshComponent::FClusterBuilder::Split(int32
Start, int32 End)
{
    // 1. 计算轴的划分方式
    // 2. 按轴排序并等分实例
    // 3. 递归处理知道Cluster大小满足要求
}
```

自底向上构建也是相似的思路，将 Cluster 视作一个节点对空间进行划分，直到每个叶子的数量为设定值则本轮划分停止

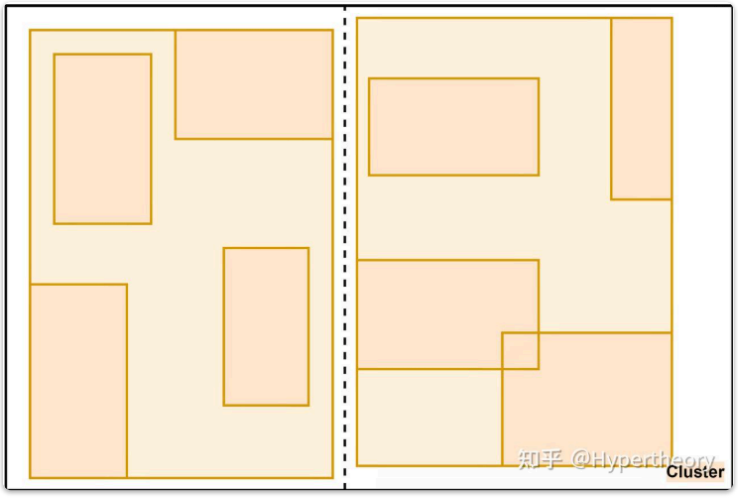

image-19.png

结点的组织形式以数组形式组织，即每个节点对应 Instance Buffer 中一段实例数据，这样在后续剔除的时候 Cluster Tree 可以保持相对的连续，减少 Instance Run

Runtime

剔除
FHierarchicalStaticMeshSceneProxy::GetDynamicMeshElements()
→FHierarchicalStaticMeshSceneProxy::Traverse()

这个方法会遍历 Cluster Tree，以下任一条件会使当前分支结束搜索进入渲染

1. 当前结点为叶子结点。
2. 当前结点的实例数量小于 `MinInstancesToSplit`，且结点内的所有实例均不会被距离剔除。
3. 当前结点的所有实例均完全包含于视锥体，且所有实例均处于同一级 `LOD`，且结点位于遮挡剔除层级或之下。